Advanced Data Handling & Manipulation

Processing Large Data Safely, Efficiently and Predictably

MEMORY EFFICIENCY

Why Advanced Data Handling Matters

- Code that works well on 1,000 rows can run out of memory, or slow to a crawl, on 100 million rows.
- Building a full list or table keeps every value in memory at once, so memory use grows with the size of the data.
- A lazy pipeline (a generator) handles one item at a time, so memory use stays almost the same whatever the size of the data.
- Vectorised tools such as pandas replace slow Python loops with a single call that runs in fast, compiled code.
- Reliable pipelines state their data types, record structure and memory limits explicitly, rather than leaving them to chance.

memory_efficiency.py

```python
import sys

def squares_list(n):
    return [x * x for x in range(n)]    # builds every value now

def squares_gen(n):
    return (x * x for x in range(n))    # builds values on request

def main():
    n = 1_000_000
    as_list = squares_list(n)
    as_gen = squares_gen(n)

    # Size of the container object itself
    print(f"List:      {sys.getsizeof(as_list):>10,} bytes")
    print(f"Generator: {sys.getsizeof(as_gen):>10,} bytes")

    # Same answer either way
    print(sum(as_list) == sum(as_gen))  # True

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- Creates the same one million squares twice: once as a list and once as a generator.
- The list holds every value at once (about 8.4 MB for the list alone); the generator object is about 200 bytes.
- Both give the same total, so the lazy version saves memory without changing the result.

MEMORY EFFICIENCY

How a Generator Builds Values on Request

- Round brackets (...) create a generator expression; square brackets [...] create a list.
- Creating a generator calculates nothing: it stores only the recipe and its current position.
- Each next() call computes one value, hands it back and pauses. for loops, sum() and list() call next() for you.
- This is why a generator stays tiny, however many values it can produce.
- A generator can be used only once. When it is exhausted, create a new one or store the values in a list.

generator_on_demand.py

```
def square(x):
    print(f"  computing {x} * {x}")
    return x * x

def main():
    # Round brackets: a lazy generator
    gen = (square(x) for x in range(3))
    print("Generator created: nothing computed yet")

    print(next(gen))     # computes 0 * 0, prints 0
    print(next(gen))     # computes 1 * 1, prints 1
    # The third value is never computed: nobody asked for it

    # Square brackets: a list computes everything at once
    print("Building a list...")
    squares = [square(x) for x in range(3)]
    print(squares)

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- No "computing" line appears when the generator is created.
- Each next() prints one "computing" line and then the value: 0, then 1. The third square is never computed.
- The list comprehension prints all three "computing" lines at once, before [0, 1, 4] is shown.

1. Generators, Iterators and Lazy Evaluation

- An iterator hands out items one at a time: iter() prepares it and next() asks for the next item.
- A generator function uses yield instead of return, pausing after each value until the next one is requested.
- A generator expression, written in round brackets, describes a calculation without running it yet.
- Avoid turning a stream into a list with list(...) unless you really need every item at the same time.
- Chain generators together to read, filter and transform large data step by step.

lazy_pipelines.py

```python
def read_numbers(path):
    """Generator: yields one number per line, never the whole file."""
    with open(path) as f:
        for line in f:
            if line.strip():
                yield int(line.strip())

def main():
    # Create a small sample file
    with open("numbers.txt", "w") as f:
        f.write("5\n12\n3\n20\n8\n15\n")

    # Generator expression: describes the work, runs nothing yet
    pipeline = (value * 2
                for value in read_numbers("numbers.txt")
                if value > 10)

    # Values are produced one at a time as the loop asks for them
    for value in pipeline:
        print(f"Processed: {value}")    # 24, 40, 30

    # Under the hood: the iterator protocol
    it = iter(read_numbers("numbers.txt"))
    print(next(it), next(it))           # 5 12

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- read_numbers reads one line, yields one number, then pauses until the next number is requested.
- pipeline keeps numbers above 10 and doubles them; nothing is computed until the for loop runs.
- Prints 24, 40 and 30; the last two lines call next() directly to show the iterator protocol behind every loop.

2. itertools for Memory-Friendly Processing

- itertools is a standard-library module of small, reusable building blocks for working with streams of data.
- islice(stream, n) takes only the first n items, like slicing a list, but without building the list.
- groupby() groups items that sit next to each other with the same key, so sort the data by that key first.
- accumulate() produces running totals (or other running results) as the data flows past.
- All three are lazy and can be combined, so a large stream is never loaded into memory in full.

itertools_processing.py

```python
from itertools import islice, groupby, accumulate

def main():
    # 1. islice: take only what you need from a stream
    stream = (x * x for x in range(100))
    first_five = list(islice(stream, 5))   # only 5 values computed
    print(f"First five squares: {first_five}")

    # 2. groupby: groups neighbouring items with the same key
    data = [
        {"category": "B", "val": 15},
        {"category": "A", "val": 10},
        {"category": "B", "val": 30},
        {"category": "A", "val": 20},
    ]
    data.sort(key=lambda row: row["category"])   # sort first!
    for key, group in groupby(data, key=lambda row: row["category"]):
        vals = [row["val"] for row in group]
        print(f"Category {key}: {vals}")

    # 3. accumulate: running totals over a stream
    daily_sales = [100, 250, 75, 300]
    print(f"Running total: {list(accumulate(daily_sales))}")

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- islice computes only the first five squares: [0, 1, 4, 9, 16]; the rest of the range is never touched.
- Sorting by category first lets groupby produce one group per category: A [10, 20] and B [15, 30].
- accumulate turns daily sales into running totals: [100, 350, 425, 725].

3. Structured Data and __post_init__ Validation

- @dataclass turns a class with typed fields into a ready-made record type: __init__, __repr__ and == are written for you.
- Type annotations such as name: str and age: int document what each field is expected to hold.
- __post_init__ runs automatically straight after an object is created, so it is the place to clean and check the data.
- A bad record raises an error immediately, at the point of entry, instead of causing confusing problems later.
- Use field(default_factory=list) for list defaults, so each object gets its own list instead of sharing one.

data_schemas.py

```python
from dataclasses import dataclass, field

@dataclass
class Customer:
    name: str
    age: int
    tags: list[str] = field(default_factory=list)  # new list per customer

    def __post_init__(self):
        """Runs straight after __init__: clean, then validate."""
        self.name = self.name.strip()
        if not self.name:
            raise ValueError("Customer name cannot be empty.")
        if self.age < 0:
            raise ValueError("Age must be zero or positive.")

def main():
    valid = Customer("  Ada Lovelace ", 36, ["pioneer"])
    print(f"Valid: {valid}")

    try:
        Customer("Bob", -5)              # invalid age
    except ValueError as err:
        print(f"Rejected: {err}")

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- Creating a Customer runs __post_init__, which trims spaces from the name and then checks the name and age.
- Customer(" Ada Lovelace ", 36, ...) is accepted and stored with the tidy name "Ada Lovelace".
- Customer("Bob", -5) is rejected with "Age must be zero or positive."

SERIALISATION

4. Portable and Safe JSON Encoding

- JSON is plain text that almost any language or service can read, but it only supports strings, numbers, lists, dictionaries, true/false and null.
- To save other types, such as datetime, subclass json.JSONEncoder and override default() to convert them to text.
- To load the data back, convert the text to the right type again and rebuild the object with Event(**data).
- Never load a pickle file from an untrusted source: unpickling can run arbitrary code on your machine.
- Use JSON for files and data shared with other systems; keep pickle for trusted, internal Python-only data.

serialization.py

```python
import json
from dataclasses import dataclass, asdict
from datetime import datetime

@dataclass
class Event:
    name: str
    timestamp: datetime          # json cannot encode this type by itself

class CustomEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()      # datetime to ISO text
        return super().default(obj)     # anything else: normal error

def main():
    event = Event("system_boot", datetime(2026, 1, 1, 9, 30))

    # Serialise: object -> dict -> JSON text
    payload = json.dumps(asdict(event), cls=CustomEncoder)
    print(f"JSON: {payload}")

    # Deserialise: JSON text -> dict -> object
    data = json.loads(payload)
    data["timestamp"] = datetime.fromisoformat(data["timestamp"])
    restored = Event(**data)
    print(f"Restored equal: {restored == event}")   # True

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- asdict(event) turns the dataclass into a dictionary; default() is then called only for the datetime field.
- Produces {"name": "system_boot", "timestamp": "2026-01-01T09:30:00"}.
- json.loads and datetime.fromisoformat rebuild an identical Event, so the final check prints True.

STREAM FILE I/O

5. Processing Large Text Files in Streams

- Reading a whole CSV file or database export into memory at once can crash the program when the file is large.
- csv.DictReader reads the file one row at a time and returns each row as a dictionary of column name to value.
- Update running totals as each row arrives, instead of storing every row first and adding them up later.
- Memory use then depends on how many results are kept (here, one total per category), not on the size of the file.
- Write results out in stages as well, so large jobs stay predictable under heavy load.

stream_file_io.py

```
import csv

def generate_csv_rows(path):
    """Yields one row (as a dict) at a time from a CSV file."""
    with open(path, newline="") as f:
        yield from csv.DictReader(f)

def main():
    # Create a small sample CSV
    with open("sales.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["category", "amount"])
        writer.writerows([["electronics", "299.99"], ["books", "15.50"],
                          ["electronics", "45.00"], ["books", "120.00"]])

    # Aggregate as rows arrive: memory depends on categories, not rows
    totals = {}
    for row in generate_csv_rows("sales.csv"):
        category = row["category"]
        totals[category] = totals.get(category, 0) + float(row["amount"])

    for category, total in totals.items():
        print(f"{category}: {total:.2f}")

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- generate_csv_rows uses yield from csv.DictReader(f) to pass rows on one at a time, straight from disk.
- Each row is added to its category total as soon as it is read, then discarded.
- Prints electronics: 344.99 and books: 135.50.

6. Vectorisation and groupby Transformations

- Looping over rows with iterrows() is slow, because every row passes through the Python interpreter one by one.
- A vectorised expression such as df["amount"] * 1.2 works on the whole column in one step, using fast compiled code.
- groupby() with named aggregations (.agg) produces one summary row per group.
- groupby().transform() returns a result for every original row, so it can be added as a new column.
- When combining tables with merge(), use validate="one_to_one" (or similar) to catch unexpected duplicates.

advanced_pandas.py

```python
import pandas as pd

def main():
    df = pd.DataFrame({
        "category": ["A", "A", "B", "B", "C"],
        "amount": [10.0, 20.0, 15.0, 35.0, 50.0],
    })

    # 1. Vectorised column arithmetic (no Python loop)
    df["amount_with_vat"] = df["amount"] * 1.2

    # 2. Named aggregation: one summary row per category
    summary = df.groupby("category", as_index=False).agg(
        total=("amount", "sum"),
        average=("amount", "mean"),
    )
    print(summary)

    # 3. transform: per-group result, aligned to every original row
    df["group_total"] = df.groupby("category")["amount"].transform("sum")
    df["share"] = df["amount"] / df["group_total"]
    print(df)

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- amount_with_vat is calculated for all five rows at once, with no Python loop.
- .agg() collapses the data to one row per category: A, B and C with their total and average.
- .transform("sum") keeps all five rows and adds each row’s category total, so share is each row’s fraction of it.

7. Narrow dtypes and Categorical Optimisation

- Text columns that repeat a few labels (such as "books" and "clothing") waste memory when every value is stored as a separate string.
- The category dtype stores each label once and keeps a small integer code for each row.
- float32 and int16 use half or a quarter of the memory of the default 64-bit types, if their range and precision are enough.
- Pass dtype= to read_csv so the narrow types are used from the moment the data is loaded.
- Use chunksize= to read a large file in pieces, then combine the partial results.

pandas_diagnostics.py

```python
import pandas as pd

def main():
    # Create a sample CSV with 400 rows
    pd.DataFrame({
        "category": ["electronics", "books", "electronics", "clothing"] * 100,
        "amount": [299.99, 15.50, 45.00, 120.00] * 100,
    }).to_csv("large_sales.csv", index=False)

    # Narrow types, applied while the file is read
    dtype_spec = {"category": "category", "amount": "float32"}

    # Read 100 rows at a time and combine the partial totals
    totals = {}
    for chunk in pd.read_csv("large_sales.csv", dtype=dtype_spec,
                             chunksize=100):
        partial = chunk.groupby("category", observed=True)["amount"].sum()
        for category, amount in partial.items():
            totals[category] = totals.get(category, 0) + float(amount)
    print({k: round(v, 2) for k, v in totals.items()})

    # Compare memory: default types vs narrow types
    wide = pd.read_csv("large_sales.csv")
    narrow = pd.read_csv("large_sales.csv", dtype=dtype_spec)
    print(wide.memory_usage(deep=True).sum(), "bytes (default types)")
    print(narrow.memory_usage(deep=True).sum(), "bytes (narrow types)")

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- Reads the 400-row file in four chunks of 100 rows and adds each chunk’s category totals together.
- Prints the combined totals for books, clothing and electronics.
- Memory check: about 26,400 bytes with default types against about 2,400 bytes with category and float32.

DIAGNOSTIC PROFILING

8. Measuring Memory Use

- tracemalloc, from the standard library, records the memory Python allocates while your code runs.
- Look at peak memory, not just current memory: the peak decides whether a job fits on the machine.
- In pandas, df.memory_usage(deep=True) shows how much memory each column really uses, including text.
- Some pandas operations return copies rather than views; unnecessary copies can double memory use.
- Measure before and after every optimisation, and keep only the changes that make a real difference.

diagnostic_profiling.py

```python
import tracemalloc

def build_list():
    return [x * 2 for x in range(500_000)]       # whole list in memory

def generator_sum():
    return sum(x * 2 for x in range(500_000))    # one value at a time

def peak_mb(func):
    tracemalloc.start()
    func()
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / (1024 * 1024)

def main():
    print(f"List peak:      {peak_mb(build_list):.2f} MB")
    print(f"Generator peak: {peak_mb(generator_sum):.3f} MB")

if __name__ == "__main__":
    main()
```

WHAT THIS CODE DOES

- peak_mb starts tracemalloc, runs a function, reads the peak memory and stops tracing.
- Building a 500,000-item list peaks at about 19 MB.
- Summing the same values with a generator peaks at almost nothing, because only one value exists at a time.

ACTIVITIES OVERVIEW

8 Activities to Consolidate Concepts

1. **Lazy Data Loader Generator** → Rewrite a list-building function so that it yields items one at a time.
2. **itertools Stream Transformations** → Use islice, groupby and accumulate on a stream without building lists.
3. **Dataclass Order Validation** → Create an Order dataclass that rejects invalid values in __post_init__.
4. **Safe Custom JSON Serialisation** → Save and reload a dataclass containing a datetime with a custom encoder.
5. **Incremental CSV Processing** → Total a large CSV file row by row with csv.DictReader.
6. **pandas Column Vectorisation** → Replace an iterrows() loop with a single column expression.
7. **pandas Memory Optimisation** → Reduce a DataFrame's memory with category and narrower numeric types.
8. **End-to-End Safe Pipeline** → Stream, validate and summarise records in one complete pipeline.